

Guide d'intégration — API Vocale Poular N'ma SIM

Ce guide explique comment intégrer la **compréhension vocale en poular** dans l'application web de la borne. Le fonctionnement est identique à l'API KYC : l'app web envoie des données, l'API renvoie un résultat JSON.

0. Origine technique

- **Extracteur** : openWakeWord (léger, tourne sur CPU)
- **Classifieurs** : RandomForest (pages service et pièce) et régression logistique (page motif)
- **Robustesse** : données augmentées + une catégorie **"rien"** qui rejette silence, bruit et parole hors-sujet
- **Seuil de confiance** : 0.35 (ajustable dans app.py)

1. Ce que fait l'API

Le client parle à la borne en poular. L'app web enregistre l'audio et l'envoie à l'API avec la **page courante**. L'API renvoie l'**intention** détectée — ou `null` si c'est du silence, du bruit, ou de la parole hors-sujet.

```
App web (borne) -- audio + page --> API Poular -- intention --> App web
```

2. Installation et lancement

Prérequis

- Python 3.10 ou 3.11
- Les fichiers du dossier `nma_vocal_api_poular/` (dont `modeles/`)

Installation

```
cd nma_vocal_api_poular
pip install -r requirements.txt
```

Lancement

```
uvicorn app:app --host 0.0.0.0 --port 8200
```

Note : le port est **8200** (l'API KYC utilise 8000, l'API soussou 8100). Ça évite les conflits si plusieurs API tournent en même temps.

Vérifier

Ouvrir `http://localhost:8200/sante` :

```
{
  "statut": "ok",
  "langue": "poular",
  "modeles_charges": ["page_choix_du_service", "page_type_de_piece", "page_motif_reactivation"],
  "modeles_manquants": []
}
```

3. L'endpoint principal : `/comprendre`

Méthode : `POST` **URL** : `http://localhost:8200/comprendre` **Format** : `multipart/form-data`

Paramètres

Champ	Type	Description
<code>page</code>	texte	La page courante
<code>audio</code>	fichier	L'audio enregistré (wav, mp3, webm, ogg... l'API convertit)

Valeurs possibles pour `page`

- `page_choix_du_service` → intentions : `nouvelle_sim` , `reactivation_sim`
- `page_type_de_piece` → intentions : `carte_electeur` , `carte_nationale_identite` , `passeport`
- `page_motif_reactivation` → intentions : `blocage` , `inactivite` , `perte`

Réponse (JSON)

```
{
  "intention": "passeport",
  "raison": "score_max",
  "scores": { "carte_electeur": 0.05, "carte_nationale_identite": 0.08, "passeport": 0.81, "rien": 0.06 },
  "niveau_sonore": 0.34,
  "duree_s": 1.6,
  "message": "Intention détectée : passeport"
}
```

Champs de la réponse

Champ	Description
<code>intention</code>	L'intention détectée, ou <code>null</code> si rien/pas compris
<code>raison</code>	<code>score_max</code> , <code>priorite_verbe_action</code> , <code>rien_detecte</code> , <code>rejet_score_faible</code> , <code>rejet_ambigu</code> , ou <code>audio_vide</code>
<code>scores</code>	Score de chaque intention (dont <code>rien</code>)
<code>niveau_sonore</code>	Niveau max de l'audio
<code>message</code>	Message lisible

4. Le cas `intention = null` (important)

Quand `intention` vaut `null` , la borne **ne doit rien lancer**. Le champ `raison` précise pourquoi :

Raison	Signification	Que faire
<code>audio_vide</code>	Silence total (micro n'a rien capté)	Continuer d'attendre
<code>rien_detecte</code>	Silence/bruit/parole hors-sujet	Continuer d'attendre
<code>rejet_score_faible</code>	Aucune intention assez claire	Demander de répéter
<code>rejet_ambigu</code>	Deux intentions trop proches	Demander de répéter

C'est le rôle de la catégorie "rien" : la borne n'agit que sur un vrai mot-clé, jamais sur du silence ou du bruit.

5. Appeler l'API depuis l'app web (JavaScript)

```
async function comprendrePoular(page, blobAudio) {
  const form = new FormData();
  form.append("page", page);
  form.append("audio", blobAudio, "audio.webm");
  const reponse = await fetch("http://localhost:8200/comprendre", {
    method: "POST",
    body: form,
  });
  return await reponse.json();
}

// Exemple :
const resultat = await comprendrePoular("page_type_de_piece", monBlobAudio);

if (resultat.intention === null) {
  // silence / bruit / pas compris -> ne rien lancer
  console.log("Pas d'action :", resultat.raison);
} else {
  // déclencher l'action correspondante
  console.log("Le client veut :", resultat.intention);
}
```

Enregistrer l'audio dans le navigateur

```
let mediaRecorder, morceaux = [];
async function demarrer() {
```

```

const flux = await navigator.mediaDevices.getUserMedia({ audio: true });
mediaRecorder = new MediaRecorder(flux); morceaux = [];
mediaRecorder.ondataavailable = e => morceaux.push(e.data);
mediaRecorder.start();
}
function arreter() {
  return new Promise(resolve => {
    mediaRecorder.onstop = () => resolve(new Blob(morceaux));
    mediaRecorder.stop();
  });
}

```

6. Brancher intention → action

Page	Intention	Action app web
page_choix_du_service	nouvelle_sim	Parcours « nouvelle SIM »
page_choix_du_service	reactivation_sim	Parcours « réactivation »
page_type_de_piece	pasport	Sélectionner passeport
page_type_de_piece	carte_nationale_identite	Sélectionner CNI
page_type_de_piece	carte_electeur	Sélectionner carte d'électeur
page_motif_reactivation	blocage / inactivite / perte	Motif correspondant

Le vocal remplace le clic : quand l'API renvoie une intention, l'app web déclenche la même chose que le bouton correspondant.

7. Réglages (dans app.py)

Deux paramètres en haut du fichier, ajustables selon les tests au micro :

- `SEUIL = 0.35` : en dessous, l'API dit "pas compris". Baisser (0.30) si de bons mots sont rejetés ; monter si des sons douteux passent.
- `MARGE_AMBIGU = 0.10` : si deux intentions sont trop proches, "ambigu". Baisser (0.05) si ça rejette trop.

La catégorie "rien" rejette le silence/bruit indépendamment du seuil (elle est apprise, pas basée sur le seuil).

8. Résumé pour démarrer vite

1. `pip install -r requirements.txt`
2. `uvicorn app:app --host 0.0.0.0 --port 8200`
3. Vérifier `http://localhost:8200/sante`
4. `POST /comprendre` avec `page + audio`
5. Lire `intention` → déclencher l'action (ou ne rien faire si `null`)

Test en ligne de commande :

```
python tester_api.py mon_audio.wav page_type_de_piece
```

API poular cohérente avec l'API KYC et l'API soussou (même principe FastAPI). Modèles avec catégorie "rien" pour rejeter le silence. *Projet N'ma SIM — OSC 2026.*